How to run the project

1. Database

Open pgAdmin or psql and run: CREATE DATABASE noc_ai;

2. Backend

```
cd noc-agentic-ai/backend
python -m venv venv
venv\Scripts\activate          (Mac/Linux: source venv/bin/activate)
pip install -r requirements.txt
copy .env.example .env          (Mac/Linux: cp .env.example .env)
```

Edit backend/.env: DATABASE_URL=postgresql://postgres:YOUR_PASSWORD@localhost:5432/noc_ai and OPENROUTER_API_KEY=sk-or-...

```
python -m app.engines.anomaly
uvicorn app.main:app --reload --port 8000
```

Open http://localhost:8000/docs to check. Tables and demo data are created on first start.

3. Frontend (new terminal)

```
cd noc-agentic-ai/frontend
npm install
copy .env.example .env.local    (Mac/Linux: cp)
npm run dev
```

Open http://localhost:5173

4. Tests

```
cd backend
pytest -v
python tests/generate_test_report.py
```

5. Deploy

• Push the folder to a new GitHub repository.

• Render: New, Blueprint, choose the repo (render.yaml creates backend + free Postgres). Add OPENROUTER_API_KEY. Copy the backend URL.

• Vercel: New Project, root directory frontend, framework Vite, env VITE_API_URL = Render URL. Deploy.

• Render free tier sleeps: open the backend URL a minute before the demo.

Demonstration script (about 12-15 minutes)

Before you start: backend and frontend running (locally or deployed), open the frontend, click **Reset** on the dashboard. Keep the dashboard visible. Optional: have http://localhost:8000/docs open in a second tab.

1. Introduce the platform (1 min)

• Dashboard: network health 100 %, no alarms. Sidebar follows the NOC workflow: Monitor, Diagnose, Restore, Knowledge.

• Network Topology: show core, aggregation, access layers. Click AR-01 and point out "If it fails, isolates: BS-101, BS-102, ENT-GW-101". Click the dashed AR-02 backup link: redundancy.

• Network Inventory: devices with vendor, interfaces, connections. (Optionally add a device.)

2. Alarm generation and ingestion (1 min)

• Dashboard > Scenario simulator > **ar01 uplink failure**.

• Say: 7 events arrive in three formats (syslog line from AR-01, JSON from base stations, one duplicate).

• Alarm Console > **All alarms**: click an alarm to show the raw vendor payload next to the normalized event.

3. Correlation (2 min)

• Dashboard: "Alarm reduction: 7 alarms -> 1 incident". Open the incident.

• Overview: primary suspected event AR-01 interface_down on GE-0/0/24, 5 consequential, 1 duplicate suppressed but kept for audit. Read the reasoning lines.

• Click **Confirm correlation** (human in the loop).

4. Topology analysis, performance, fault localization (2 min)

• Performance tab: anomaly model flagged link_down on 4 devices; before vs current table; the 5 investigation questions.

• Fault tab: AR-01 high confidence, the evidence list (uplink interface, earliest alarm, explains 4/4 devices, peers healthy), alternatives with reasons.

5. Service impact (1 min)

• Impact tab: 3 services unavailable, 7,300 known customers and "unknown" for E-101: the system never invents customer counts.

6. Root cause hypotheses + historical incidents + procedures (2 min)

• RCA tab: ranked hypotheses, each with supporting AND contradicting evidence and required verification. Stress "suspected" vs "confirmed".

• Similar historical incident INC-NET-032 (fiber uplink failure, 3 sites) used as evidence only.

• Knowledge Base page: ask "What troubleshooting procedure applies to this alarm?" - answer cites SOP-001 sections.

7. Restoration recommendation and human approval (2 min)

• Plan tab: diagnostics first, then disruptive (red), then escalation; each step cites an SOP section.

• Action Tracker: try **Start** on the shut/no-shut action -> blocked: needs approval.

• Approval Center: approve diagnostic steps and the shut/no-shut; show **Modify** (edited text is re-classified) and **Reject**.

• Action Tracker: Start -> Complete with a result.

8. Dynamic reassessment (1 min)

• While the incident is still active: simulator > **new alarm during incident** (CR-01 side of the uplink goes down).

• Incident page shows the yellow notice: revision 2, "Fault location changed: AR-01 -> L-CR01-AR01" (both link ends down now points to the fiber span).

• Agent Execution: pick the new run; show it re-ran all agents.

9. Post-restoration validation and closure (1 min)

• Incident > **Simulate restoration + validate**: baseline / during / after table, alarms cleared, PASS, status resolved.

• Enter confirmed root cause "Fiber cut on CR-01 - AR-01 span" > **Confirm root cause** > **Close incident**.

• **Download PDF** report: 16 sections.

10. Agent visibility + other scenarios (2 min)

• Agent Execution: pipeline strip, every input, tool call (topology.localize, anomaly_model.score, search_procedures(RAG)), LLM call, messages between agents, critic review, human decisions. Show agent prompts at the bottom.

• Reset, then run **alarm storm**: Alarm Console cluster view: ~206 alarms -> 2 incidents (AR-03 storm + unrelated BS-201 power failure).

• Run **congestion**: no device alarm at all - the anomaly model itself raised the incident; the investigation shows congestion supported but the RCA keeps it as a hypothesis.

• Performance page: model evaluation table (precision / recall / F1 for Isolation Forest, z-score, hybrid).

Likely interview questions (short answers)

• *Why deterministic tools + LLM?* Facts (topology, scores, impact) must be correct and reproducible; the LLM reasons over them, may move confidences by at most 0.2 and never supplies evidence. If the free model fails, every agent has a rule-based fallback.

• *Why Isolation Forest + z-score?* No labelled real data; per-device baselines make one model serve all device types; z-score keeps it explainable; IF catches unusual metric combinations.

• *How is a storm handled?* O(n) duplicate pass, time-sorted union-find with early break on the window, cached topology relations, summarized by device/type.

• *How is safety ensured?* Classification safety net on keywords, critic agent re-check with rework loop, API blocks execution of unapproved disruptive actions, no code path touches devices.